Design Track

Interview Preparation Series • Professional Edition

Message Queues & Streaming

Compiled with LaTeX • June 30, 2026

harsha.somisetty01@gmail.com

In This Edition

> **How to use this file:** Read deeply once to build mechanism-level intuition (not just vocabulary), trace each diagram by hand, then use the Revision Cheat Sheet before interviews. The goal is to be able to *design* an async pipeline end-to-end — topics, partitioning, idempotency, DLQs, ordering, scaling — and defend every trade-off under follow-up pressure.

1 Overview --- What It Is

A **message queue** (more broadly, **asynchronous messaging middleware**) is infrastructure that lets one component (a **producer**) hand off work or facts to another component (a **consumer**) *without waiting for the consumer to finish — or even to be online*. The middleware durably holds the message in between.

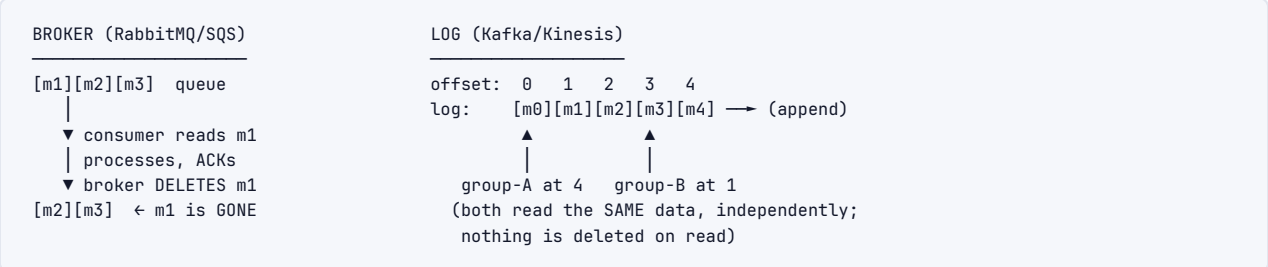
That single sentence hides three separate guarantees that are the whole reason the category exists:

1. **Temporal decoupling** — producer and consumer do not need to be available at the same instant. The broker buffers.
2. **Spatial decoupling** — the producer does not need to know who the consumers are, how many there are, or where they run. It addresses a *destination* (a queue or topic), not a *peer*.
3. **Durability of in-flight work** — once the broker accepts (ACKs) a message, it survives consumer crashes, restarts, and (with replication) broker crashes.

There are three structurally different shapes this takes, and confusing them is the #1 vocabulary mistake candidates make:

Shape	Canonical example	One-line mental model
Work queue	RabbitMQ queue, AWS SQS	A to-do list. Each task goes to *exactly one* worker; once done (ACKed) it's **deleted**.
Pub/Sub	SNS, RabbitMQ fanout exchange	A loudspeaker. Each event is broadcast to *every* interested subscriber.
Distributed log	Apache Kafka, AWS Kinesis	An append-only ledger. Events are **retained** and **replayable**; consumers track their own position (offset).

The defining distinction of the log model (Kafka/Kinesis) vs the broker model (RabbitMQ/SQS): in a broker, reading is *destructive* (delete-on-ack); in a log, reading is *non-destructive* (the consumer just advances a cursor, the data stays). This one property cascades into everything: replay, multiple independent consumer groups, retention, stream processing, event sourcing.



Interview takeaway: ”A queue is a to-do list you cross items off; a log is a ledger you keep. The moment you say 'I need to replay events' or 'two independent teams consume the same stream,' you've chosen a log (Kafka), not a queue (SQS/RabbitMQ).”

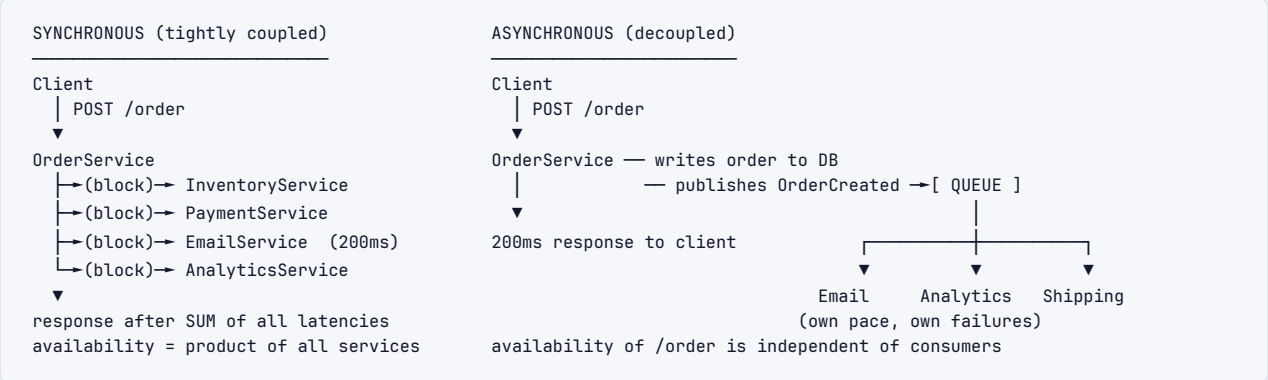
2 Why It Exists

Asynchronous messaging exists to break the **synchronous coupling** that makes naive distributed systems fragile, slow, and impossible to scale independently. Walk through the failure modes it removes.

2.1 Sync vs Async Trade-offs

In a **synchronous** call, the caller blocks until the callee responds. This is simple and gives you an immediate answer — but it welds the two services together:

- **Availability is multiplicative.** If `OrderService` synchronously calls `EmailService`, `InventoryService`, and `AnalyticsService`, the order endpoint is up only when *all four* are up. With 99.9% each, end-to-end availability $\approx 0.999^4 \approx$ **99.6%** ($\approx$ 35 hours/year down vs 8.7 hours for one service). Every synchronous dependency you add *lowers* your availability.
- **Latency is additive.** The user waits for the *sum* of all downstream latencies. Sending a welcome email (200 ms) should not be on the critical path of "place order."
- **Failure cascades.** If `EmailService` gets slow, threads in `OrderService` pile up waiting on it, the thread pool exhausts, and `OrderService` falls over too — even though placing an order has nothing to do with email. This is the cascading-failure pattern.



When sync is correct: the caller genuinely needs the answer *now* to proceed (a price quote, an auth check, a "is this username taken"). Async is wrong there — you'd be inventing a request/response protocol on top of a queue, badly.

When async is correct: the work can happen *after* you respond to the user (notifications, indexing, analytics, fan-out to many consumers, anything that absorbs load spikes).

2.2 Decoupling (the deeper point)

Decoupling is not just "don't block." It's that the **producer has no compile-time or runtime knowledge of consumers.** You can add a new consumer (say, a fraud-detection service that also wants `OrderCreated` events) *without touching the producer or redeploying it.* In a synchronous design, adding a fifth downstream call means editing and redeploying `OrderService`. This is why event-driven architectures scale *organizationally* — teams add consumers independently. (Conway's Law in action.)

2.3 Load Leveling (Spike Absorption)

A queue acts as a **shock absorber** between a bursty producer and a fixed-capacity consumer. The producer can spike to 10,000 req/s; the consumers drain at a steady 1,000 msg/s; the queue depth rises, then falls. Nothing is dropped, nothing is overloaded — the spike is *spread over time*.

```
SPIKE ABSORPTION (numbers)

Incoming rate (Black Friday flash sale):
   t=0s   500/s   ▒▒▒▒
   t=10s  9000/s  ████████████████████  ← 18x normal
   t=30s  9000/s  ████████████████████
   t=60s  500/s   ▒▒▒▒

Consumer capacity: 1000 msg/s (fixed — 10 workers × 100 msg/s)

WITHOUT a queue:  9000/s hits the DB directly → DB connection pool (say 200)
                  saturates → timeouts → cascading 500s → outage.

WITH a queue:
   Queue depth over time (in = 9000/s for 20s, out = 1000/s):
     accumulated backlog at t=30s ≈ (9000-1000)×20 = 160,000 messages
     drain time after spike      ≈ 160,000 / 1000  = 160s
   Result: users get a fast 202 Accepted; the backlog drains over ~3 min;
           the DB never sees more than 1000 writes/s. No outage.
```

The cost of load leveling is **latency under load**: during the spike, a message might sit in the queue for up to ~160 s before processing. You trade tail latency for survival. For order *confirmation emails* that's fine; for *fraud blocking before charge* it may not be — which tells you which work belongs async.

2.4 Back-Pressure

The flip side of load leveling: what happens when consumers *permanently* can't keep up (not a transient spike)? The queue grows **unboundedly**, you run out of disk/memory, and eventually the broker dies — taking everyone with it. **Back-pressure** is the set of mechanisms that push the ”slow down” signal back toward the source:

- **Bounded queues / lag alerts:** Kafka can't bound a topic (it's a log), but you alarm on **consumer lag**. RabbitMQ supports `x-max-length` queues that drop or dead-letter overflow.
- **Pull-based consumption (Kafka/SQS):** consumers *pull* at their own rate; they simply ask for less. A push broker (classic RabbitMQ) must use **prefetch (QoS)** limits so it doesn't shove more at a consumer than it can handle.
- **Publisher throttling / rejection:** when the queue is over a high-water mark, reject new publishes (RabbitMQ's flow control, returning 429/503 to the API layer). This propagates pressure all the way to the client, who retries with backoff.
- **Reactive Streams / TCP-style credit:** consumer advertises how much it can take; producer never exceeds that credit.

```
Producer ──(too fast)──► [ Queue grows ] ──► Consumer (saturated)
                                │
        BACK-PRESSURE signal ◄──┘  "slow down / I'll pull when ready / queue full → 503"
```

Interview takeaway: ”A queue absorbs *spikes* (temporary) via load-leveling, but it cannot absorb a *sustained* overload — that just moves the failure from 'consumer falls over' to 'broker fills up.' The fix is back-pressure: pull-based consumers, prefetch limits, lag alarms, and publisher rejection that propagates the slow-down back to the client.”

3 Why FAANG Cares

Compa	What they run	What they probe in interviews
Amazon	SQS and SNS are *foundational* AWS services; internal order/fulfillment pipelines are async; Kinesis for streaming	Async decoupling, SQS visibility timeout & DLQs, SNS→SQS fan-out, exactly-once *processing* with idempotency, the dual-write/outbox problem
Linked	**Invented Kafka.** Their entire data backbone is Kafka (activity tracking, metrics, CDC, stream processing with Samza)	Kafka partitions/consumer groups/ISR cold; log compaction; exactly-once; schema evolution with their Avro schema registry
Meta	Scribe (log aggregation), Wormhole (pub/sub CDC), LogDevice (distributed log)	High-fan-out delivery, ordering guarantees, at-least-once + idempotency, backfill/replay
Netflix	Kafka (Keystone pipeline, trillions of events/day), SQS, Flink for stream processing	Stream processing, windowing, back-pressure, DLQ + retry strategies, multi-region
Google	Cloud Pub/Sub, Dataflow (Apache Beam), internal pub/sub	Exactly-once *processing*, event-time vs processing-time, watermarks, windowing
Uber	Massive Kafka deployment, Flink for real-time pricing/ETA	Ordering per key (per-rider/per-trip), consumer lag, exactly-once, dead-letter handling

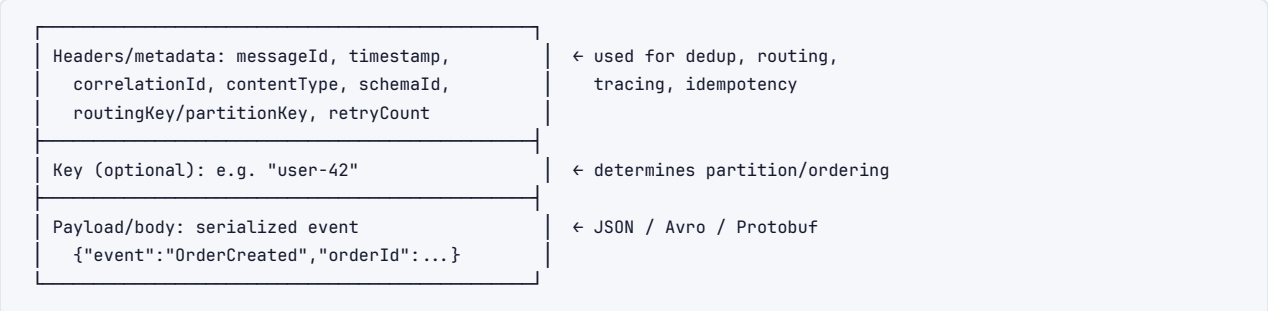
Universal signal: can you (1) decide *whether* a piece of work should be async, (2) choose the right shape (queue vs log), (3) reason about delivery semantics and make consumers idempotent, and (4) name the failure modes (poison messages, dual writes, ordering loss, lag) *before being asked*? Saying "I'll just throw it on a queue" without addressing duplicates, ordering, and DLQs is the junior tell.

4 Core Concepts

4.1 Producer, Broker, Consumer

- **Producer (publisher):** creates messages and sends them to a destination (queue/topic/exchange). Cares about: did the broker durably accept this? (the **ack/confirm**).
- **Broker:** the server (cluster) that receives, stores, routes, and delivers messages. Provides durability (disk + replication), routing, and delivery tracking.
- **Consumer (subscriber):** reads messages and does work. Cares about: only mark "done" *after* the work succeeds (the **consumer ack/commit**).

4.2 Message Anatomy



Two header fields earn their keep in every serious design: a **messageId/idempotency key** (for dedup) and a **key/partition key** (for ordering). Memorize that.

4.3 ACK, NACK, and Redelivery

The consumer ack is the heart of reliability. The lifecycle:

```
1. Broker delivers msg to consumer (marks it "in-flight / unacked")
2. Consumer processes it
3a. Success → ACK   → broker deletes (queue) / consumer commits offset (log)
3b. Failure → NACK/reject → broker REDELIVERS (requeue) or dead-letters
3c. Consumer crashes before ACK → broker times out the in-flight msg → REDELIVERS
```

The crucial subtlety: **when does the in-flight message become visible again?** In SQS it's the **visibility timeout**. In RabbitMQ it's *when the channel/connection drops* (or on explicit nack). In Kafka there's no per-message ack at all — the consumer periodically **commits an offset** marking "I've processed up to here."

4.4 Offset / Cursor (log model only)

In a log, each consumer (group) maintains an **offset** per partition: the position of the next message to read. This is *the* difference from a broker. Because the offset is just a number the consumer controls, you can:

- **Replay**: reset offset to 0 (or a timestamp) and reprocess history.
- **Skip**: jump past a poison message.
- **Run multiple independent consumers**: each group has its own offsets over the same retained data.

4.5 Dead Letter Queue (DLQ)

A side queue where messages go after exhausting retries (a "poison message" that keeps failing). Lets you (a) stop blocking the main queue, (b) inspect/alert/replay later. Every production design must have one.

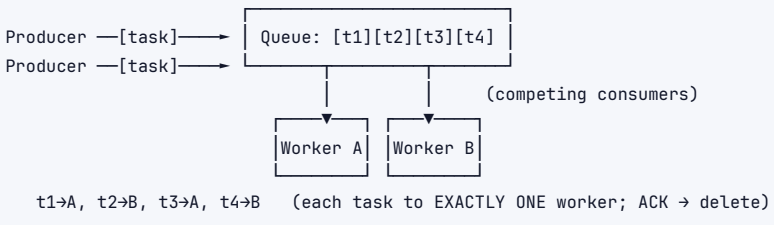
4.6 Consumer Group

A set of cooperating consumers that *share* the work of a topic/queue. In Kafka, each partition is assigned to exactly one consumer in the group (that's the parallelism unit). In SQS, multiple consumers polling the same queue *are* effectively a group (the queue load-balances). The key invariant: **within a group, a message is processed once; across groups, every group sees everything** (log model).

5 Queue vs Pub/Sub vs Log

This is the most important conceptual section. Get the semantics exactly right.

5.1 Point-to-Point Queue (Work Queue)

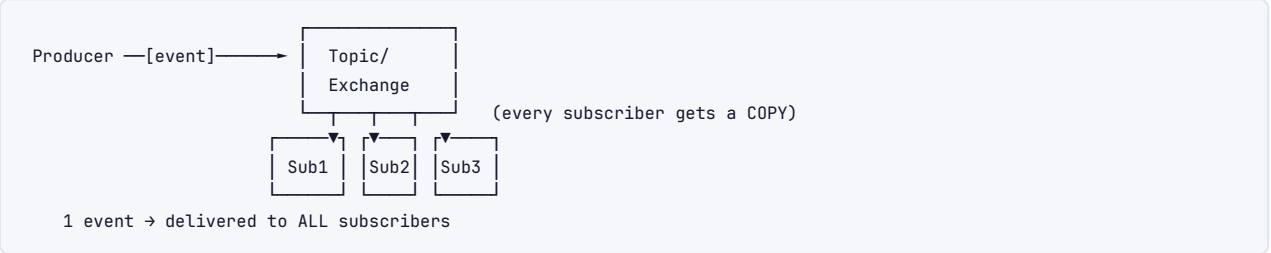


Semantics: competing consumers. The broker distributes each message to *one* of the available workers, then **deletes** it on ack. Adding workers = more throughput (they split the work).

This is the *task distribution* model.

- **Use when:** "process this job once" — image resizing, sending one email, charging one card, a background job.
- **Brokers:** RabbitMQ (default queue), SQS, Celery/Sidekiq backends.

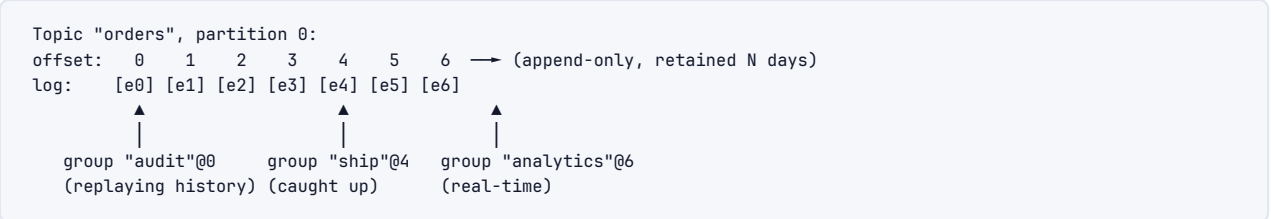
5.2 Publish/Subscribe (Fan-out)



Semantics: broadcast. Each subscriber gets its *own copy* of every message. Adding a subscriber does *not* reduce what others get — it's a new independent recipient. This is the *event notification* model.

- **Use when:** one fact, many interested parties — `OrderCreated` should notify shipping, analytics, email, and fraud, each independently.
- **Brokers:** SNS, RabbitMQ fanout/topic exchange, Kafka with multiple consumer groups, Google Pub/Sub, Redis Pub/Sub.

5.3 Distributed Log (Retained, Replayable)



Semantics: the log is *both* a queue (within one consumer group, partitions are split across consumers → work queue behavior) *and* pub/sub (multiple groups each see everything) — **plus retention and replay.** The data is not deleted on read; it's deleted by a **retention policy** (time/-size) or **compaction**.

- **Use when:** you need replay, event sourcing, multiple independent downstreams, stream processing, CDC, or very high throughput.
- **Brokers:** Kafka, Kinesis, Pulsar, Redpanda.

5.4 Broker-Centric vs Log-Centric --- Deep Comparison

Dimension	Broker-centric (RabbitMQ, SQS)	Log-centric (Kafka, Kinesis)
Read model	Destructive — delete on ack	Non-destructive — advance offset
Storage	Holds until consumed (then gone)	Retains for a window (hours→forever) regardless of consumption
Replay	No (it's gone)	Yes — reset offset / seek by time
Multiple independent consumers	Need a separate queue/binding per consumer	Free — each group has its own offsets

Dimension	Broker-centric (RabbitMQ, SQS)	Log-centric (Kafka, Kinesis)
Ordering	Per-queue (FIFO variants); broken by competing consumers + requeue	Strict **per-partition**
Routing	Rich (exchanges, routing keys, headers)	Dumb — partition by key hash; routing logic lives in consumers/stream apps
Throughput	High (tens–hundreds of K/s)	Very high (millions/s) — sequential disk, zero-copy, batching
Per-message ops	Per-message ack/nack/priority/TTL/delay	No per-message ack; coarse offset commits; no priorities
Mental model	Smart broker, dumb consumer	Dumb broker, smart consumer
Backlog visibility	Queue depth	Consumer lag (end offset − committed offset)

The crisp heuristic:

- Need **complex routing**, per-message TTL/priority/delay, classic task queues, or RPC-style request/reply? → **RabbitMQ** (smart broker).
- Need **replay, retention, many independent consumers, event sourcing, CDC, stream processing, extreme throughput**? → **Kafka** (smart consumer, dumb broker).
- Want **zero ops, AWS-native, simple decoupling**? → **SQS** (queue) / **SNS** (fan-out) / **Kinesis** (when you need log semantics on AWS).

Interview takeaway: "RabbitMQ is a smart broker with a dumb consumer — routing logic lives in the broker. Kafka is a dumb broker with a smart consumer — the broker just appends to a log; consumers own their offsets and any routing. That single architectural choice is why Kafka replays and RabbitMQ doesn't."

6 Delivery Semantics

There are three theoretical guarantees. Understanding *which is achievable and why* is a top FAANG discriminator.

6.1 At-Most-Once

Message delivered zero or one times. Never duplicates; **may lose** messages.

- **How:** fire-and-forget. Producer doesn't wait for broker ack (`acks=0`); or consumer **commits the offset before processing** (so a crash mid-process loses the message).
- **Use when:** loss is acceptable and duplicates are costly — high-volume metrics, some logging, sampled telemetry.

6.2 At-Least-Once

Message delivered one or more times. Never loses; **may duplicate**.

- **How:** producer retries until it gets a broker ack (`acks=all` + retries); consumer **commits the offset / acks only AFTER processing** succeeds. If the consumer crashes after processing but before committing, the message is redelivered → duplicate.
- **Use when:** the default for almost everything, *paired with idempotent consumers*. This is the workhorse.

6.3 Exactly-Once

Delivered and processed precisely once. The holy grail — and **end-to-end exactly-once *delivery* is provably impossible** across an unreliable network with crashes.

Why Exactly-Once *Delivery* Is Impossible (Two Generals)

The **Two Generals Problem**: two generals must agree to attack at the same time, coordinating only via messengers who may be captured (lost messages). General A sends "attack at dawn." Did it arrive? A needs an ACK. B sends an ACK. Did *that* arrive? B now needs an ACK-of-the-ACK. This regresses infinitely — **no finite protocol can guarantee both sides know with certainty** over a lossy channel.

Map this to messaging: a consumer processes a message, then must tell the broker "done" (ack). If the **ack is lost** (network drop, consumer crash *right after* processing but *before* the ack lands), the broker cannot distinguish "consumer processed it and the ack was lost" from "consumer died before processing." Its only safe choices:

- **Redeliver** → risk a duplicate (at-least-once).
- **Don't redeliver** → risk a loss (at-most-once).

There is *no third option* for *delivery*. You cannot get exactly-once delivery. Full stop.

```
Consumer: process(msg) ──► send ACK ──X──► (ACK LOST)
Broker: "I never heard back. Did it process or die?"
   redeliver → DUPLICATE   |   give up → LOSS
   ↑ no way to know which is correct → exactly-once DELIVERY impossible
```

What *Is* Achievable: At-Least-Once + Idempotent Consumer = Effectively-Once

Since you can't prevent duplicate *delivery*, you make duplicate *processing* harmless. The consumer detects "I've already handled this message" and no-ops. This is **effectively-once / exactly-once *processing*** — and it's what virtually every real system does.

Python

```python
# At-least-once delivery + idempotent consumer via a dedup table.
# The dedup insert and the business write happen in ONE DB transaction,
# so they commit or roll back atomically.

def handle(msg, db):
    msg_id = msg.headers["messageId"]   # producer-assigned unique id
    with db.transaction() as tx:
        try:
            # Unique constraint on processed_messages(message_id) is the guard.
            tx.execute(
                "INSERT INTO processed_messages(message_id, processed_at) "
                "VALUES (%s, now())",
                (msg_id,),
            )
        except UniqueViolation:
            # Already processed → this is a redelivery. No-op, ack, done.
            return ACK

        # First time we've seen this id → do the real work in the SAME tx.
        apply_business_effect(tx, msg)     # e.g. credit account, create order
        # tx commits: dedup row + business effect together, atomically.
    return ACK
```

```sql
CREATE TABLE processed_messages (
    message_id   UUID PRIMARY KEY,        -- the idempotency guard
    processed_at TIMESTAMPTZ NOT NULL
);
-- TTL/cleanup: periodically delete rows older than the broker's max
-- redelivery window (e.g. retention + visibility timeout). You only need
-- to remember an id as long as it could possibly be redelivered.
```

Why the transaction matters: if you `INSERT` the dedup row in one transaction and do the business effect in another, a crash between them gives you a recorded-but-not-applied message (loss) or applied-but-not-recorded (future duplicate). One transaction makes "remembered" and "applied" the same event.

Exactly-Once *Processing* via Kafka Transactions

Kafka offers true exactly-once *within the Kafka boundary* by combining:

1. **Idempotent producer** — each producer gets a Producer ID (PID) and per-partition sequence numbers; the broker drops duplicate batches caused by producer retries (so `acks=all` retries don't create duplicates).
2. **Transactions** — a producer can atomically write to multiple partitions/topics *and commit the consumer offsets* in one transaction. The classic **consume-process-produce** loop becomes atomic: "read from input topic, transform, write to output topic, commit input offsets" either all happens or none does.

```java
producer.initTransactions();
while (true) {
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofMillis(100));
    producer.beginTransaction();
    for (ConsumerRecord<String,String> r : records) {
        ProducerRecord<String,String> out = transform(r);
        producer.send(out);                       // write to output topic
    }
    // Commit the INPUT offsets as part of the SAME transaction:
    producer.sendOffsetsToTransaction(offsetsOf(records), consumer.groupMetadata());
    producer.commitTransaction();                 // all-or-nothing
}
// Consumers of the output topic set isolation.level=read_committed
// so they never see records from aborted transactions.
```

The big caveat: this is exactly-once only *inside Kafka* (Kafka → Kafka). The moment your consumer writes to an *external* system (a SQL DB, an HTTP API, sending an email), Kafka transactions don't cover it — you're back to at-least-once across that boundary and you need the **idempotent-consumer / dedup-table** pattern (or to enroll the external write in the same DB transaction as the offset).

6.4 Ack Timing --- The Loss vs Duplicate Lever

Where you place the offset commit / ack relative to processing *chooses your semantics*:

```
PATTERN A — commit BEFORE processing (at-most-once):
  read msg → COMMIT offset → process
  crash after commit, before process → message LOST (offset already advanced)

PATTERN B — commit AFTER processing (at-least-once):  ☑ the right default
```

```
read msg → process → COMMIT offset
crash after process, before commit → message REDELIVERED → DUPLICATE
(harmless if consumer is idempotent)
```

```python
# At-least-once: commit only after the side effect is durable.
for msg in consumer:
    process_and_persist(msg)     # do the work, make it durable
    consumer.commit(msg.offset)  # only NOW advance the cursor
    # If we crash between the two lines, msg is redelivered. Idempotency saves us.
```

Interview takeaway: "Exactly-once *delivery* is impossible — the Two Generals Problem proves the ACK can always be lost, forcing a choice between redeliver (duplicate) or not (loss). So you pick at-least-once (commit *after* processing) and make the consumer idempotent. That yields exactly-once *processing*, which is what everyone actually means. Kafka transactions give true exactly-once, but only Kafka-to-Kafka."

7 Kafka Deep Dive

Kafka is the most-probed messaging system in senior interviews. Know it mechanism-deep.

7.1 Topics and Partitions

A **topic** is a named stream. It's split into **partitions** — each partition is an independent, ordered, append-only log on disk. Partitions are *the* fundamental unit of two things at once:

- **Parallelism:** N partitions ⇒ up to N consumers in a group can read in parallel. Partition count is your max consumer parallelism.
- **Ordering:** order is guaranteed *only within a partition*. Across partitions there is no global order.

```
Topic "orders" (3 partitions, replication-factor=3)

Partition 0: [o0][o3][o6][o9]...   (an ordered log)
Partition 1: [o1][o4][o7]...
Partition 2: [o2][o5][o8]...

Producer chooses partition:
  • key present → partition = hash(key) % numPartitions   (sticky per key)
  • key null    → round-robin / sticky batching across partitions
```

Partition-by-key is the linchpin of ordering: if all events for `orderId=42` use key `"42"`, they hash to the *same* partition, so they're strictly ordered relative to each other. Events for different orders may interleave across partitions — and that's fine, because they're independent.

Choosing partition count: more partitions = more parallelism + throughput, but also more open file handles, more memory, longer leader-election/rebalance times, and worse end-to-end latency. You can *increase* partitions later but it **breaks key→partition mapping** (existing keys rehash to different partitions, scrambling ordering for in-flight data). So over-provision modestly up front. Rule of thumb: target a partition per expected concurrent consumer, sized so each handles a manageable MB/s.

7.2 Consumer Groups and Rebalancing

A **consumer group** shares a topic's partitions: each partition is owned by exactly one consumer in the group at a time. If you have 3 partitions and 3 consumers, each gets one. With 4

consumers, one sits idle (more consumers than partitions = wasted consumers). With 2 consumers, one handles 2 partitions.

```
Topic: 3 partitions     Group "billing":
  P0 ──► Consumer A
  P1 ──► Consumer B
  P2 ──► Consumer C
Consumer C dies → REBALANCE → P2 reassigned to A or B.
```

Rebalancing reassigns partitions when consumers join/leave or partitions change. The catch:

- **Eager (stop-the-world) rebalancing** (older default): *all* consumers revoke *all* partitions, then everyone gets new assignments. Processing halts entirely during the rebalance — a latency spike for the whole group. Bad if rebalances are frequent.
- **Cooperative / incremental rebalancing** (Kafka 2.4+): only the partitions that actually need to move are revoked; the rest keep processing. Much less disruptive.
- **Static membership** (`group.instance.id`): a consumer that restarts within `session.timeout.ms` keeps its assignment, avoiding a rebalance on rolling restarts entirely.

A common operational pain: a slow consumer misses heartbeats / exceeds `max.poll.interval.ms`, gets kicked from the group, triggers a rebalance, and the redelivered messages cause duplicates. Tune `max.poll.records` and processing time, or move heavy work off the poll thread.

7.3 Offsets: Committed vs Current, Auto vs Manual

- **Current/position offset:** where the consumer will read next (advances as it polls).
- **Committed offset:** the last offset durably saved (to the internal `__consumer_offsets` topic) as "processed." On restart/rebalance, the consumer resumes from the committed offset.
- **Lag** = log-end-offset − committed-offset = how far behind you are.

```
Partition 0:  ...[e97][e98][e99][e100][e101][e102]
                           ▲committed          ▲log-end (latest)
                           position=e100
              lag = 102 - 99 = 3 messages behind
```

- **Auto-commit** (`enable.auto.commit=true`): commits the *current* position every `auto.commit.interval.ms`. Dangerous — it can commit offsets for messages you *polled* but haven't *finished processing*, so a crash loses them (silent at-most-once-ish). Convenient but a footgun.
- **Manual commit** (`enable.auto.commit=false`): you call `commitSync()` / `commitAsync()` *after* processing. This is the correct choice for at-least-once. Commit after the batch is durably handled.

7.4 Replication, ISR, Leader Election

Each partition has one **leader** and `replication.factor - 1` **followers** on other brokers. All reads/writes go to the leader; followers replicate from it.

- **ISR (In-Sync Replicas):** the set of replicas (including the leader) that are caught up with the leader (within `replica.lag.time.max.ms`). A replica that falls behind drops out of the ISR; when it catches up, it rejoins.
- **Leader election:** if the leader broker dies, a *new leader is elected from the ISR* (because only in-sync replicas are guaranteed to have all committed data). If you allow `unclean.leader.election=true`, an out-of-sync replica *can* become leader — restoring availability but **losing data** (it's missing recent writes). Default is `false`: prefer durability over availability.

```
Partition 0, replication.factor=3:
  Broker1 (LEADER) ──► Broker2 (follower, in ISR) ──► Broker3 (follower, lagging → OUT of ISR)
  ISR = {B1, B2}
  B1 dies → new leader elected from ISR = B2 (has all committed data). B3 cannot win (clean election).
```

7.5 `acks` and `min.insync.replicas` --- The Durability Dial

Producer `acks` controls how many replicas must confirm a write before the producer considers it successful:

`acks`	Waits for	Durability	Latency	Semantics
`0`	Nothing (fire-and-forget)	Lowest — message can vanish	Lowest	At-most-once
`1`	Leader only	Medium — lost if leader dies before a follower replicates	Medium	Can lose on leader failover
`all` (`-1`)	All in-sync replicas	Highest — survives leader loss	Highest	Basis for at-least-once

`acks=all` alone isn't enough. If the ISR has shrunk to *just the leader* (followers lagging), `acks=all` means "ack from 1 replica" — no real redundancy. **`min.insync.replicas`** sets the floor: with `min.insync.replicas=2` and `acks=all`, a write is rejected (producer gets `NotEnoughReplicas`) unless at least 2 replicas are in sync. The durable-config triad:

```python
replication.factor = 3
min.insync.replicas = 2     (tolerate 1 broker down and still accept writes)
acks = all                  (wait for all ISR members, ≥ 2)
```

This survives one broker failure with zero data loss and keeps accepting writes. Lose two brokers → writes are *rejected* (CP choice: refuse rather than risk loss).

7.6 Log Retention and Compaction

Kafka deletes data by policy, not by consumption.

- **Time/size retention** (`cleanup.policy=delete`): keep the last `retention.ms` (e.g., 7 days) or `retention.bytes`. Old segments are deleted wholesale. This is for *event streams*.
- **Log compaction** (`cleanup.policy=compact`): keep *at least the latest value per key*, deleting older values for the same key. This turns a topic into a **changelog / materialized state** — replay it to rebuild a key→latest-value table. Used for CDC, Kafka Streams state stores, and `__consumer_offsets` itself.

```
LOG COMPACTION (cleanup.policy=compact)

Before:  k=A v=1 | k=B v=1 | k=A v=2 | k=C v=1 | k=B v=2 | k=A v=3
                             ─────              ─────     ─────
After:                       k=C v=1 | k=B v=2 | k=A v=3
(only the LATEST value per key survives; a tombstone `k=X v=null` deletes a key)
```

7.7 Exactly-Once (Idempotent Producer + Transactions)

Covered in Delivery Semantics — `enable.idempotence=true` (default in modern Kafka) gives per-partition dedup via PID+sequence; `transactional.id` + `read_committed` consumers gives atomic consume-process-produce. Remember: **Kafka-to-Kafka only.**

7.8 Throughput Tuning

Kafka is fast because of sequential disk writes, the OS page cache, **zero-copy** (`sendfile` — disk→socket without copying to user space), and **batching**. Producer knobs:

- **`batch.size`** — max bytes per partition batch. Bigger = fewer requests, more throughput, more latency.
- **`linger.ms`** — how long to wait to fill a batch before sending. `linger.ms=0` = send immediately (low latency); `linger.ms=20` = wait up to 20 ms to batch (high throughput). The core latency-vs-throughput dial.
- **`compression.type`** — `lz4`/`zstd`/`snappy`. Compresses the *batch* (so bigger batches compress better). Trades CPU for network/disk and storage.
- **`max.in.flight.requests.per.connection`** — with idempotence on, can be up to 5 while preserving ordering; without idempotence, >1 risks reordering on retry.

```java
props.put("acks", "all");
props.put("enable.idempotence", true);
props.put("linger.ms", 20);          // batch for up to 20ms
props.put("batch.size", 65536);      // 64KB batches
props.put("compression.type", "zstd");
```

7.9 Consumer Lag

The single most important Kafka health metric. **Lag = end offset − committed offset**, per partition. Rising lag = consumers can't keep up (need more consumers up to #partitions, or faster processing, or more partitions). Flat-high lag during a spike is normal load-leveling; *monotonically rising* lag is an outage forming. Alert on it.

7.10 Full Topic Diagram

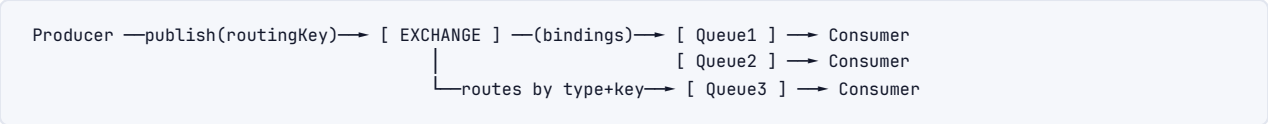
```
Topic "user-events"  (4 partitions, replication-factor=3, retention=7d)

Brokers:    B1            B2            B3
Part 0:   LEADER       follower      follower      ISR={B1,B2,B3}
Part 1:   follower     LEADER        follower      ISR={B2,B3}
Part 2:   follower     follower      LEADER        ISR={B1,B3}
Part 3:   LEADER       follower      follower      ISR={B1,B2}

Producers (key = userId):
  userId=10 → hash → P2     userId=11 → hash → P0     userId=12 → hash → P3

Consumer Group "analytics" (3 consumers):
  Consumer X → P0, P3      Consumer Y → P1      Consumer Z → P2
  (4 partitions, 3 consumers → one consumer owns two partitions)

Consumer Group "fraud" (2 consumers):   ← independent group, own offsets
  Consumer P → P0, P1      Consumer Q → P2, P3
  (sees EVERY message independently of "analytics")

Rules:
  • each partition → exactly ONE consumer per group
  • ordering guaranteed WITHIN a partition (all of userId=10's events ordered)
  • each group commits its own offsets → independent positions & replay
```

8 RabbitMQ Deep Dive

RabbitMQ implements **AMQP 0-9-1** — a "smart broker" with rich routing. The model has more moving parts than Kafka but gives you per-message control.

8.1 The AMQP Model: Exchanges → Bindings → Queues

Producers **never publish to queues directly.** They publish to an **exchange** with a **routing key**. The exchange uses **bindings** (rules) to route the message into zero or more **queues**. Consumers read from queues.



8.2 Exchange Types

Type	Routing logic	Example
Direct	Deliver to queues whose binding key *exactly equals* the routing key	routingKey `"order.created"` → queue bound with `"order.created"`
Topic	Wildcard match: `*` = one word, `#` = zero+ words	binding `"order.*.eu"` matches `"order.created.eu"`; `"order.#"` matches `"order.created.eu.priority"`
Fanout	Ignore the key — broadcast to *all* bound queues (pub/sub)	event bus: every bound queue gets a copy
Header	Match on message *header* attributes instead of routing key	`x-match=all` with `{type:order, region:eu}`

Python

```
TOPIC EXCHANGE example
publish routingKey = "order.created.eu"

bindings:
  Queue "all-orders"     ← "order.#"          ☑ matches
  Queue "eu-orders"      ← "order.*.eu"       ☑ matches
  Queue "created-orders" ← "order.created.*"  ☑ matches
  Queue "us-orders"      ← "order.*.us"       ☐ no match
```

Python

```
# Direct exchange + ack/nack + prefetch (pika)
ch.exchange_declare(exchange="orders", exchange_type="direct", durable=True)
ch.queue_declare(queue="order-created", durable=True)
ch.queue_bind(queue="order-created", exchange="orders", routing_key="order.created")

# Persistent message (survives broker restart) — durability needs BOTH
# durable queue AND persistent message (delivery_mode=2).
ch.basic_publish(
    exchange="orders", routing_key="order.created",
    body=payload,
    properties=pika.BasicProperties(delivery_mode=2),
)
```

8.3 ACK / NACK / Reject and Requeue

The consumer must acknowledge. Options:

- **`basic.ack`** — processed successfully → broker deletes it.
- **`basic.nack`** / **`basic.reject`** — failed. `requeue=true` → put it back (retry); `requeue=false` → drop or send to the **DLX** (dead-letter exchange).

- **No ack + connection drops** — broker assumes failure and **requeues** to another consumer (this is how crashes are handled).

```python
def on_message(ch, method, props, body):
    try:
        process(body)
        ch.basic_ack(delivery_tag=method.delivery_tag)        # success
    except TransientError:
        ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)   # retry
    except PoisonError:
        ch.basic_nack(delivery_tag=method.delivery_tag, requeue=False)  # → DLX
```

Requeue caution: `requeue=true` on a poison message creates a hot loop — it's redelivered instantly, fails again, forever, pinning a CPU. Use a **retry-with-delay + max-attempts → DLX** strategy instead (see Reliability Patterns).

8.4 Prefetch (QoS) --- Consumer Back-Pressure

By default RabbitMQ *pushes* messages to consumers as fast as it can — a slow consumer gets buried. **`basic.qos(prefetch_count=N)`** caps the number of *unacked* messages a consumer may hold at once. With `prefetch=1`, the broker won't send a second message until the first is acked → perfect fair dispatch for uneven task durations. Higher prefetch = more throughput but risks one consumer hoarding work.

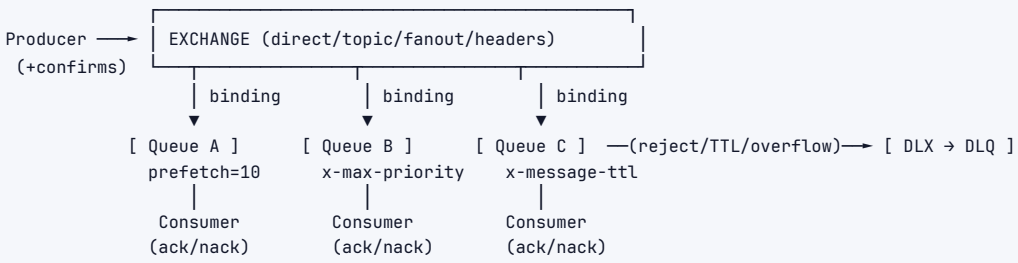
```python
ch.basic_qos(prefetch_count=10)   # at most 10 in-flight unacked per consumer
```

8.5 Dead Letter Exchange (DLX), TTL, Priority, Confirms

- **DLX:** a queue declared with `x-dead-letter-exchange` sends messages there when they're rejected (`requeue=false`), expire (TTL), or exceed `x-max-length`. The DLQ is for inspection/replay.
- **TTL** (`x-message-ttl`): messages expire after N ms → dead-lettered. Combine with DLX to build **delayed retry**: publish to a "wait" queue with TTL=30s and DLX pointing back to the work queue → message reappears after 30s.
- **Priority queues** (`x-max-priority`): higher-priority messages jump ahead. (Kafka has *no* priorities — a real RabbitMQ advantage.)
- **Publisher confirms** (`confirm_select`): the broker asynchronously acks the *producer* once the message is safely persisted/replicated. This is RabbitMQ's `acks=all` equivalent — without it, a publish that the broker drops is silently lost.

```python
ch.confirm_select()                       # enable publisher confirms
if not ch.basic_publish(..., mandatory=True):
    raise PublishFailed                   # broker did not confirm → retry
```

8.6 AMQP Model Summary Diagram



9 Cloud Brokers (SQS/SNS/Kinesis)

The managed AWS trio — zero ops, pay-per-use, the default for AWS-native designs.

9.1 SQS --- Standard vs FIFO

A fully managed queue (work-queue / point-to-point model). Two flavors:

	Standard	FIFO
Ordering	Best-effort (mostly ordered, not guaranteed)	Strict, per **MessageGroupId**
Delivery	At-least-once (rare duplicates)	Exactly-once *processing* (5-min dedup)
Throughput	Nearly unlimited	300 msg/s (3,000 with batching) per group
Dedup	None (you handle it)	Content-based or explicit `MessageDeduplicationId`
Use	Default, high throughput	Order matters & duplicates unacceptable

Visibility timeout — the core SQS mechanism. When a consumer receives a message, SQS makes it **invisible** to other consumers for the visibility timeout (default 30 s). The consumer processes, then calls `DeleteMessage` (the ack). If it deletes in time → done. If it crashes or runs long → the timeout expires → the message reappears → redelivered (at-least-once). Set the timeout > your p99 processing time, or extend it with `ChangeMessageVisibility` for long jobs.

```
t=0   receive(msg) → SQS hides msg for 30s (visibility timeout)
t=5   consumer crashes (no DeleteMessage)
t=30  timeout expires → msg visible again → another consumer receives it (redelivery)
      — if instead: t=10 DeleteMessage → msg gone, no redelivery
```

Long polling (`WaitTimeSeconds=20`): instead of returning instantly empty (short polling → wasteful, costly, many empty receives), the `ReceiveMessage` call waits up to 20 s for a message to arrive. Fewer empty responses, lower cost, lower latency. Always use long polling.

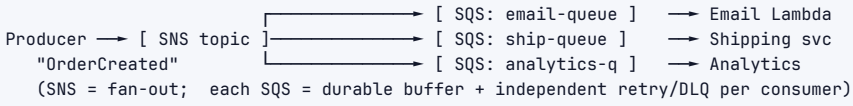
```python
resp = sqs.receive_message(QueueUrl=url, WaitTimeSeconds=20, MaxNumberOfMessages=10)
for m in resp.get("Messages", []):
    process(m)
    sqs.delete_message(QueueUrl=url, ReceiptHandle=m["ReceiptHandle"])  # ack
```

SQS DLQs: configure a `RedrivePolicy` with `maxReceiveCount` (e.g., 5). After 5 failed receives (message keeps reappearing), SQS moves it to the DLQ automatically.

9.2 SNS --- Fan-out (Pub/Sub)

Simple Notification Service is the **pub/sub** half. Publish once to an SNS **topic**; SNS pushes a copy to every **subscription** (SQS queues, Lambda, HTTP endpoints, email, SMS).

The canonical AWS pattern: SNS → SQS fan-out. One event must reach several independent consumers, each with its own retry/DLQ/back-pressure. SNS fans out to multiple SQS queues; each consumer owns a queue.



Why SNS→SQS rather than SNS→Lambda directly? The SQS in the middle gives each consumer **durability and back-pressure** — if the analytics worker is down, its queue buffers; the email path is unaffected. SNS alone would drop or retry-and-give-up.

9.3 Kinesis --- Managed Log (Shards)

Kinesis Data Streams is AWS's **Kafka analog** — a retained, replayable log. The partition unit is a **shard** (≈ Kafka partition):

- Each shard: ~1 MB/s or 1,000 records/s ingest; 2 MB/s read.
- **Partition key** → hashed → shard (same per-key ordering as Kafka).
- Retention: 24 h default, up to 365 days. Replay by sequence number / timestamp.
- **Consumers** read per-shard; with **enhanced fan-out**, each consumer gets dedicated 2 MB/s/shard throughput (vs shared standard).
- Scale by **resharding** (split/merge shards) — analogous to changing Kafka partitions, with the same key-remapping caveat.

SQS vs Kinesis (a favorite question): SQS = queue, delete-on-read, no replay, one logical consumer set, auto-scales throughput. Kinesis = log, retained, replayable, many independent consumers, fixed shard throughput you provision. "Need replay or multiple independent readers? Kinesis. Just decouple work? SQS."

10 Ordering & Idempotency

10.1 Ordering Is Per-Partition Only

The hard truth: **no scalable broker gives total global order across a topic.** Global order requires a single serialization point (one partition / one queue / one consumer) — which kills parallelism. So every system gives **ordering within a partition/group only**:

- Kafka: ordered within a partition.
- Kinesis: ordered within a shard.
- SQS FIFO: ordered within a `MessageGroupId`.
- RabbitMQ: ordered within a queue *with a single consumer* (competing consumers + requeue break it).

10.2 Partition-by-Key

To get the ordering you *actually* need, route all causally-related events to the same partition via a **key**. You don't need *global* order — you need order *per entity*:

- All events for one **order** → key = `orderId` → same partition → strictly ordered.
- All events for one **account** → key = `accountId`.

- All events for one **chat conversation** → key = `conversationId`.

Different orders/accounts/chats interleave freely across partitions — and that's correct, because they're independent. You preserve the order that matters while keeping full parallelism *across* keys.

```
Events: O1.create, O2.create, O1.pay, O1.ship, O2.cancel
key = orderId:
  P0: [O1.create][O1.pay][O1.ship]     ← O1 strictly ordered ✓
  P1: [O2.create][O2.cancel]           ← O2 strictly ordered ✓
  (O1 vs O2 interleaving is irrelevant — independent entities)
```

10.3 The Global-Ordering-Kills-Parallelism Trade-off

If you genuinely need total order, you must use **one partition** → **one consumer** → throughput capped at a single consumer's rate, no horizontal scaling. This is occasionally required (a global sequence of ledger entries), but it's a deliberate, expensive choice. The senior move is to question whether you *really* need global order or just per-entity order (almost always the latter).

```
Global order: [1 partition] → [1 consumer]   throughput = 1 consumer  (no scaling)
Per-key order: [N partitions] → [N consumers] throughput = N consumers (scales)
                                              order preserved per key, not globally
```

10.4 Out-of-Order Handling

Even with keys, things arrive out of order across partitions, on replay, or after retries. Strategies:

- **Sequence numbers / versions:** each event carries a monotonic version; the consumer ignores any event with version ≤ what it has applied (last-write-wins by version). Makes reordering safe.
- **Event-time buffering / watermarks:** in stream processing, buffer briefly and reorder by event timestamp before emitting (see Stream Processing).
- **Idempotent + commutative operations:** design so order doesn't matter (set-based merges, CRDTs).

10.5 Idempotency (Mechanism Recap)

An operation is idempotent if doing it twice equals doing it once. Because at-least-once *will* deliver duplicates, idempotency is mandatory:

- **Idempotency key + dedup table** (the dedup-table code in Delivery Semantics) — the general-purpose answer.
- **Natural idempotency:** `SET status='PAID'` (idempotent) instead of `balance = balance - 100` (not). Upsert by primary key instead of insert.
- **Conditional writes / optimistic concurrency:** `UPDATE ... WHERE version = 7` — a duplicate with stale version no-ops.

Interview takeaway: "You never need global ordering; you need per-entity ordering, which you get by partitioning on the entity key. And because at-least-once delivers duplicates, every consumer must be idempotent — by a dedup table, a natural set-operation, or a version-conditional write."

11 Reliability Patterns

The patterns that separate a toy queue demo from a production pipeline.

11.1 Idempotent Consumers

Already covered (dedup table / natural idempotency / conditional writes). The non-negotiable foundation — *every* at-least-once consumer needs it.

11.2 DLQ + Retry with Exponential Backoff + Jitter

A transient failure (downstream blip) should be **retried**; a permanent failure (malformed message) should be **dead-lettered**. Retrying immediately and forever is wrong on both counts: it hammers a struggling downstream and loops forever on poison.

- **Exponential backoff:** wait 1s, 2s, 4s, 8s... — give the downstream room to recover, don't thundering-herd it.
- **Jitter:** add randomness so that N consumers retrying don't all fire at the same instant (synchronized retries re-create the spike). Full jitter: `sleep = random(0, base * 2^attempt)`.
- **Max attempts → DLQ:** after K attempts, stop and dead-letter for human inspection / later replay.

```python
import random

MAX_ATTEMPTS = 5
BASE = 1.0  # seconds

def consume(msg):
    attempt = int(msg.headers.get("retryCount", 0))
    try:
        process(msg)
        ack(msg)
    except TransientError:
        if attempt + 1 ≥ MAX_ATTEMPTS:
            send_to_dlq(msg)          # exhausted → DLQ for inspection/replay
            ack(msg)                  # remove from main queue
            return
        # exponential backoff with FULL JITTER
        delay = random.uniform(0, BASE * (2 ** attempt))   # 0..1, 0..2, 0..4, ...
        msg.headers["retryCount"] = attempt + 1
        schedule_retry(msg, delay)    # re-publish to a delay queue / set visibility
        ack(msg)                      # remove current copy
    except PoisonError:
        send_to_dlq(msg); ack(msg)    # permanent → straight to DLQ, no retries
```

In practice: SQS uses `maxReceiveCount` + redrive to DLQ; RabbitMQ uses a TTL "wait" queue + DLX to back off then retry; Kafka uses a retry-topic chain (`orders.retry.5s`, `orders.retry.30s`, ... → `orders.DLT`) à la Spring Kafka.

```
RETRY TOPIC CHAIN (Kafka)
orders ──fail──► orders.retry.5s ──(5s later)──► reprocess ──fail──► orders.retry.30s
                                                                          │ fail
                                                                          ▼
                                                                  orders.DLT (dead-letter)
```

11.3 Poison Messages

A **poison message** can never be processed (corrupt payload, deleted referenced entity, schema mismatch). Without a max-retry/DLQ cutoff it blocks the partition/queue forever (a "head-of-line block" — everything behind it stalls). The DLQ exists precisely to *get the poison out of the way* so the rest of the stream flows, while preserving the bad message for debugging. Always cap retries.

11.4 The Dual-Write Problem + Outbox Pattern

The dual-write problem is the single most important reliability pitfall in event-driven systems. A service often needs to do two things atomically: (1) update its database, and (2) publish an event. But the DB and the broker are *separate systems* — you cannot write to both in one transaction.

```
PROBLEM:
  tx.commit(order)        ☑ committed to DB
  broker.publish(event)   ☒ broker down / process crashes here
  → DB has the order, but NO event ever fired. Downstreams never learn. (LOST EVENT)

  OR the reverse:
  broker.publish(event)   ☑ event fired
  tx.rollback(order)      ☒ DB write failed
  → event fired for an order that DOESN'T EXIST. (PHANTOM EVENT)
```

Naively wrapping them ("publish then commit" or "commit then publish") *cannot* fix this — there's always a crash window between the two. The robust solution is the **Transactional Outbox**:

1. In the *same DB transaction* as the business write, insert a row into an `outbox` table.
2. A separate **relay** (poller or CDC) reads the outbox and publishes to the broker, marking rows as sent.

Because the order and the outbox row commit in one transaction, they're atomic — you can't have one without the other. The relay then guarantees at-least-once publish (it retries until the broker acks), and consumers dedup.

SQL

```sql
BEGIN;
  INSERT INTO orders(id, user_id, total, status) VALUES (...);
  INSERT INTO outbox(id, aggregate_id, type, payload, created_at, sent)
         VALUES (gen_random_uuid(), :orderId, 'OrderCreated', :json, now(), false);
COMMIT;   -- order + outbox row commit ATOMICALLY (one DB, one tx)
```

Python

```python
# RELAY: poll the outbox and publish (at-least-once; consumers dedup).
def relay_loop(db, broker):
    while True:
        rows = db.query(
            "SELECT * FROM outbox WHERE sent = false ORDER BY created_at LIMIT 100 "
            "FOR UPDATE SKIP LOCKED")            # SKIP LOCKED → run multiple relays safely
        for r in rows:
            broker.publish(topic=r.type, key=r.aggregate_id, value=r.payload)
            db.execute("UPDATE outbox SET sent = true WHERE id = %s", (r.id,))
        sleep(0.2)
```

CDC variant (preferred at scale): instead of polling, use **Change Data Capture** — Debezium tails the database's write-ahead log (binlog/WAL) and streams every committed row change

(including outbox inserts) into Kafka. No polling lag, no extra DB load, exactly-once-ish from the log position. This is how LinkedIn/Netflix-scale systems do it.

```
APP ──tx──► [ orders + outbox ] (Postgres)
                  │ WAL
                  ▼
            Debezium (CDC) ──► Kafka topic "OrderCreated" ──► consumers (idempotent)
```

11.5 The Inbox Pattern

The consumer-side mirror of the outbox. Before processing, record the incoming `messageId` in an **`inbox`** table (unique constraint) *in the same transaction* as the side effect. On redelivery, the insert fails the unique constraint → skip. This is the dedup-table pattern formalized — "outbox guarantees you *publish* once-effectively; inbox guarantees you *consume* once-effectively."

11.6 Saga Pattern (Distributed Transactions)

When a business operation spans multiple services (each with its own DB), you can't use a single ACID transaction or 2PC (too slow, blocks on coordinator failure). A **Saga** breaks it into a sequence of local transactions, each publishing an event; if a step fails, **compensating transactions** undo the prior steps. You get *eventual* consistency, not atomicity.

```
Order Saga (happy path):
  CreateOrder → ReserveInventory → ChargePayment → ScheduleShipping → ConfirmOrder

If ChargePayment FAILS, run compensations in reverse:
  ReleaseInventory ← CancelOrder
```

Choreography vs Orchestration:

	Choreography	Orchestration
Coordination	None central — each service listens for events and reacts, emitting the next event	A central **orchestrator** tells each service what to do next
Coupling	Loose, fully event-driven	Services coupled to the orchestrator
Flow visibility	Hard — the logic is smeared across services ("what's the flow?")	Easy — one place holds the state machine
Failure/compe	Each service must know its own compensation triggers	Orchestrator drives compensations explicitly
Best for	Few steps, simple flows	Many steps, complex branching, need observability
Risk	Cyclic event dependencies, hard to debug	Orchestrator is a SPOF/bottleneck (mitigate: durable state machine — AWS Step Functions, Temporal)

```
CHOREOGRAPHY                          ORCHESTRATION
Order ─OrderCreated─► Inventory       Orchestrator ──► Order
Inventory ─Reserved─► Payment              │   ◄──events──┘
Payment ─Charged─► Shipping               ├──► Inventory
(each step listens & emits)               ├──► Payment
                                          └──► Shipping
                                      (central state machine drives + compensates)
```

11.7 Schema Registry + Avro/Protobuf Compatibility

Producers and consumers evolve independently. If a producer adds a field and an old consumer chokes, you've broken production. A **schema registry** (Confluent Schema Registry, AWS Glue) stores versioned schemas; producers register a schema and write only the schema **ID** + compact binary (Avro/Protobuf, not JSON). Consumers fetch the schema by ID to deserialize, and the registry **enforces compatibility** on registration:

- **Backward compatible:** new schema can read old data (e.g., adding a field with a default). New consumers, old data. → safe to upgrade consumers first.
- **Forward compatible:** old schema can read new data (e.g., adding optional fields old readers ignore). Old consumers, new data. → safe to upgrade producers first.
- **Full:** both.

Avro/Protobuf over JSON: smaller (no field names on the wire), faster, and *schema-enforced* — you can't accidentally ship a breaking change.

Interview takeaway: "The two failure modes people forget: the **dual-write problem** (DB and broker can't share a transaction → use the Outbox + CDC), and **schema evolution** (producer/consumer drift → use a schema registry with compatibility checks). Mentioning these unprompted is a strong senior signal."

12 Stream Processing

Beyond moving messages, you often need to *compute* over the stream continuously — aggregations, joins, enrichment. This is **stream processing** (Kafka Streams, Apache Flink, Spark Structured Streaming, Beam/Dataflow).

12.1 Kafka Streams vs Flink (Basics)

- **Kafka Streams:** a *library* (no separate cluster) that runs in your app, reads/writes Kafka topics, and keeps local state (RocksDB) backed by compacted **changelog topics** for fault tolerance. Great for Kafka-to-Kafka transforms, joins, aggregations. Scales by adding instances (rebalance like a consumer group).
- **Apache Flink:** a *distributed cluster* with true streaming, sophisticated state, event-time processing, exactly-once via **checkpoints** (periodic consistent snapshots of all operator state to durable storage; on failure, restore from the last checkpoint). The choice for complex, large-scale, low-latency stateful processing (Uber pricing, Netflix). Heavier to operate.

12.2 Stateless vs Stateful Processing

- **Stateless:** each record processed independently — `filter`, `map`, `enrich-by-lookup`. Trivial to scale and recover (no state to lose).
- **Stateful:** the result depends on prior records — counts, sums, joins, windows, deduplication. Requires durable, partitioned state and fault-tolerant recovery (changelog/checkpoints). This is where the hard problems live.

12.3 Windowing

Unbounded streams have no "end," so aggregations operate over **windows** of time:

```python
TUMBLING (fixed, non-overlapping):   [0-5)[5-10)[10-15)   each event in exactly ONE window
                                     |____|____|____|     e.g. "count per 5-min bucket"

SLIDING / HOPPING (overlapping):     [0-5)                each event in MULTIPLE windows
                                       [2-7)               e.g. "5-min count, updated every 2 min"
                                         [4-9)

SESSION (activity-gap based):        [== burst ==]  gap   [== burst ==]
                                     window closes after N idle minutes; per-key
                                     e.g. "user session = activity until 30-min idle"
```

Window	Shape	Use
Tumbling	Fixed, contiguous, no overlap	Per-interval metrics (”orders per minute”)
Sliding/Hoppin	Fixed size, overlapping by a step	Moving averages, ”last 5 min refreshed every 1 min”
Session	Dynamic, closes after inactivity gap	User sessions, bursty per-entity activity

12.4 Event Time vs Processing Time, and Watermarks

The deep problem of stream processing:

- **Event time:** when the event *actually happened* (timestamp in the event — e.g., the click happened at 12:00:03).
- **Processing time:** when your system *processes* it (e.g., it arrived at 12:00:09 due to a mobile network delay).

These diverge because of network delays, retries, offline buffering, and out-of-order delivery. If you window by *processing time*, a late-but-important event lands in the wrong bucket → wrong analytics. Correct systems window by **event time**.

But event-time windows raise a question: when is a window ”complete” if late events can still trickle in? You can't wait forever. The answer is a **watermark** — a heuristic assertion ”I believe I've now seen all events up to event-time T.” When the watermark passes a window's end, the window is *closed and emitted*. Events arriving after the watermark are **late** and handled by policy (drop, side-output, or update via allowed-lateness).

```
event time:  12:00:00 ──────────────────────────► 12:00:10
arrivals (processing order):  03, 01, 05, 04, [09 arrives but ts=02 ← out of order]
watermark advances:  "seen everything up to 12:00:04" → close window [00-05)
   the ts=02 event arriving AFTER the watermark = LATE → drop or side-output
```

Interview takeaway: ”The hard part of stream processing isn't the aggregation — it's *time*. Process by **event time**, not arrival time, and use **watermarks** to decide when a window is complete and how to treat late events. Fault tolerance for the resulting state comes from changelog topics (Kafka Streams) or checkpoints (Flink).”

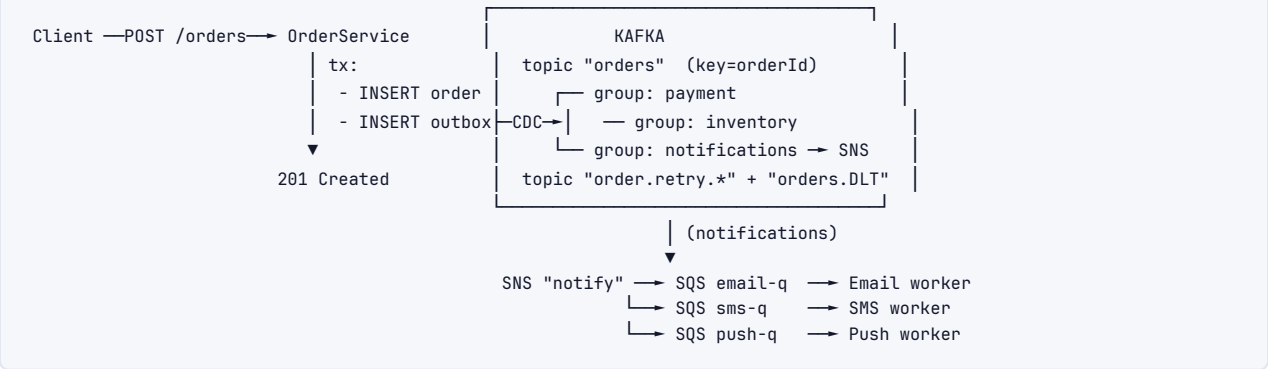
13 Designing an Async Pipeline (Worked Example)

Prompt: Design the backend for **order processing + notifications** at an e-commerce site. When a user places an order we must: persist it, reserve inventory, charge payment, schedule shipping, and notify the user (email/SMS/push) — reliably, at scale, without blocking the checkout response.

13.1 1. What's sync vs async?

- **Sync (on the critical path):** validate cart, check inventory availability (a quick read), authorize payment *hold* (user must know immediately if the card is declined), persist the order. Respond `201 Created` fast.
- **Async (off the path):** capture/settle payment, decrement inventory, schedule shipping, send notifications, update analytics/search index, fraud scoring. None of these should make the user wait.

13.2 2. Topology



13.3 3. Why these choices

- **Kafka for the core event backbone:** we need *multiple independent consumers* (payment, inventory, notifications, fraud, analytics) of the *same* `OrderCreated` event, plus **replay** (reprocess a day's orders after a bug) and **retention** (audit). That's the log model → Kafka, not SQS.
- **Outbox + CDC** on `OrderService`: persisting the order and emitting `OrderCreated` must be atomic — the **dual-write problem**. We write the order and an `outbox` row in one DB transaction; Debezium (CDC) tails the WAL and publishes to Kafka. No lost or phantom events.
- **SNS→SQS fan-out for notifications:** email/SMS/push are independent side-effects with different rate limits and failure profiles. SNS broadcasts; each SQS queue gives that channel its own buffer, retry, and DLQ. If the SMS provider is down, its queue backs up without affecting email.

13.4 4. Partitioning & ordering

- `orders` topic keyed by **`orderId`** → all events for one order (`Created`, `PaymentCaptured`, `Shipped`) are strictly ordered within a partition. Different orders parallelize across partitions.
- **Partition count:** size for peak. Say peak 5,000 orders/s, a consumer handles ~500/s → ~10–12 partitions (round up to leave headroom; over-provision modestly since increasing later rescrambles keys). Replication factor 3, `min.insync.replicas=2`, `acks=all`.

13.5 5. Delivery semantics & idempotency

- **At-least-once** everywhere: producers `acks=all`+idempotent; consumers commit offsets *after* processing.
- **Idempotent consumers** are mandatory — redelivery *will* happen. The **payment** consumer is the scariest: a duplicate must not double-charge.

```python
def handle_order_created(event, db):
    order_id = event["orderId"]
    with db.transaction() as tx:
        # inbox/dedup: charge at most once per order
        try:
            tx.execute("INSERT INTO charged_orders(order_id) VALUES (%s)", (order_id,))
        except UniqueViolation:
            return ACK                      # already charged → no-op
        # also pass an idempotency key to the payment gateway as a second guard
        payment_gateway.charge(
            idempotency_key=f"order-{order_id}",
            amount=event["total"], card=event["cardToken"],
        )
    return ACK
```

Two layers of protection: our `charged_orders` dedup table *and* the payment gateway's own `idempotency_key`. Belt and suspenders for money.

13.6 6. Reliability: DLQ, retries, poison

- **Transient failures** (payment gateway 503, inventory service timeout) → **exponential backoff + jitter** via retry topics (`orders.retry.5s` → `orders.retry.1m` → ...), max 5 attempts → `orders.DLT`.
- **Poison messages** (malformed event, deleted product) → straight to `orders.DLT`, alert on-call, never block the partition.
- A DLT consumer/dashboard lets ops inspect and **replay** fixed messages back onto `orders`.

13.7 7. Saga for the cross-service transaction

Reserve-inventory → charge-payment → schedule-shipping spans three services. Use an **orchestration saga** (durable state machine, e.g. Temporal/Step Functions) because the flow has branches and we want visibility + clean compensation:

```
ReserveInventory → ChargePayment → ScheduleShipping → ConfirmOrder
   if ChargePayment fails → compensate: ReleaseInventory, CancelOrder, NotifyUser
```

13.8 8. Notifications: ordering & dedup

- Order doesn't matter much across channels, but a user shouldn't get *two* "order confirmed" emails on redelivery → notification workers dedup on `(orderId, channel, templateId)`.
- SNS→SQS gives each channel independent back-pressure; SQS DLQ (`maxReceiveCount=5`) catches a permanently failing address.

13.9 9. Scaling & operations

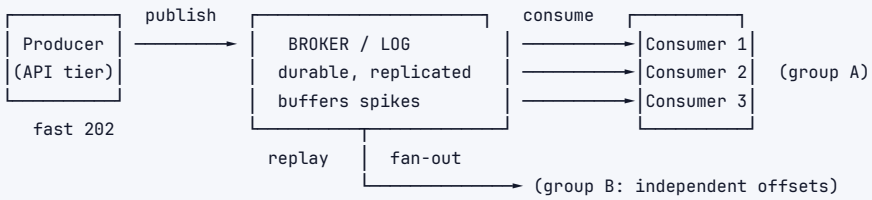
- **Scale consumers** up to #partitions; if you need more parallelism, add partitions (accepting the key-remap cost) or split topics.
- **Monitor consumer lag** per group — rising lag on `payment` = add consumers or the gateway is slow. Alarm before the backlog becomes user-visible.
- **Schema registry** with backward compatibility so producers can add fields without breaking the fraud/analytics consumers that lag behind on deploys.
- **Multi-AZ** brokers; `unclean.leader.election=false` (we'd rather reject a write than lose a paid order).

13.10 10. Recap of the patterns used

Outbox + CDC (dual-write), at-least-once + idempotent consumers (delivery), partition-by-key (ordering), DLQ + backoff/jitter (poison/transient), SNS→SQS fan-out (independent consumers), saga (distributed transaction), schema registry (evolution), consumer-lag monitoring (back-pressure). That's the full senior checklist applied to one problem.

14 Architecture / Diagrams

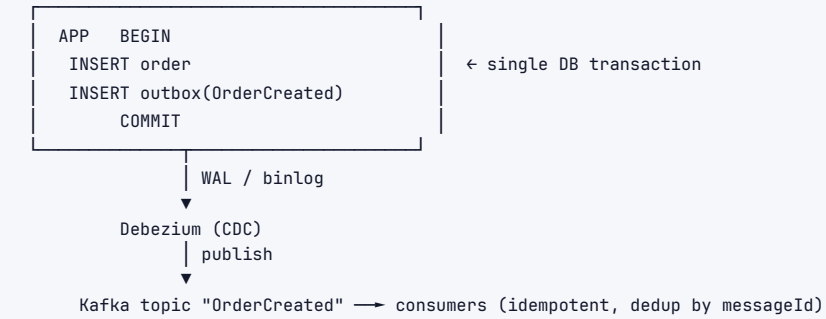
14.1 Async Decoupling (the canonical picture)



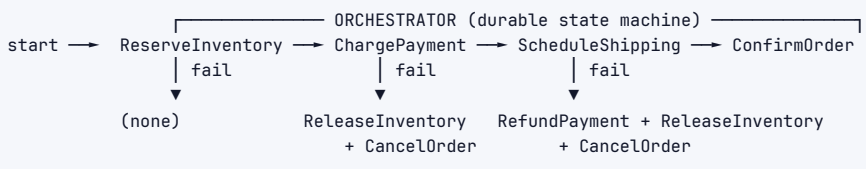
14.2 Kafka vs RabbitMQ Mental Models

```
KAFKA (dumb broker, smart consumer)        RABBITMQ (smart broker, dumb consumer)
log: append-only, retained                 exchange routes → queues (delete on ack)
consumer owns offset → replay              broker pushes; prefetch limits in-flight
ordering: per partition                    ordering: per queue (1 consumer)
routing: hash(key)→partition               routing: direct/topic/fanout/headers
```

14.3 Outbox + CDC Flow



14.4 Saga (Orchestration) with Compensation



14.5 Retry + DLQ Pipeline

15 Real-World Examples

System	Tech	Pattern / Notes
LinkedIn activity tracking	Kafka (origin)	Every page view/click → Kafka → Samza/Flink → search, analytics, recommendations. Replay for new ML features.
Uber dispatch/pricing	Kafka + Flink	Per-trip ordering (key=tripId); Flink computes real-time surge pricing/ETA with event-time windows.
Netflix Keystone	Kafka (trillions/day) + Flink	Ingest events → route to S3/Elasticsearch/analytics; back-pressure and DLQ at massive scale.
Amazon order pipeline	SQS + SNS + Kinesis	Decoupled fulfillment; SNS→SQS fan-out; visibility timeout + DLQ; idempotent workers.
Stripe payments	Internal queues + idempotency keys	Every API call takes an `Idempotency-Key`; retries never double-charge — textbook idempotent consumer.
Robinhood / trading	Kafka	Per-account ordering (key=accountId); outbox for order-event consistency; strict at-least-once + dedup.
Slack messaging	Kafka + per-channel ordering	key=channelId → message order preserved per channel; fan-out to all members' clients.
CDC pipelines (everywhere)	Debezium → Kafka	DB changes streamed as events → cache invalidation, search indexing, data lake, microservice sync.

16 Real-Life Analogies

One bustling town and its mail/dispatch systems — every concept is a counter, a courier, or a ledger.

Concept	Analogy
Message Queue	The town's parcel depot. You drop a parcel and leave; the depot holds it until the right courier is free to deliver. You never wait at the counter for the recipient to receive it.
Decoupling	You address parcels to ”the bakery,” not to ”Maria who works Tuesdays.” The bakery can change staff, hire ten more bakers, or close on Sundays — your parcel still arrives, and you never had to know who'd handle it.
Load leveling	On festival day a thousand parcels flood in at noon; the depot's six couriers deliver steadily through the evening. The rush is *spread over hours* instead of trampling the couriers all at once.
Back-pressure	When the depot's shelves are full, the front desk stops accepting new parcels and tells senders ”come back in an hour” — the ”full” signal travels back to the senders so the depot never collapses under its own backlog.
Queue vs Pub/Sub	A *work queue* is the depot's job board — each delivery slip is taken by exactly one courier and torn down once done. *Pub/sub* is the town crier — he shouts the news once and *every* shopkeeper hears their own copy.

Concept	Analogy
Log (Kafka)	The town's permanent ledger of every event, written in ink and never erased. Each reader keeps a bookmark (offset); two historians can read the same ledger at their own pace, and anyone can flip back to re-read last year.
Delete-on-ack vs retained	The job board *tears down* a slip when the courier reports done (it's gone). The ledger *keeps* every entry; readers just move their bookmark forward — the ink stays.
At-least-once + idempotency	A nervous courier, unsure the first delivery registered, delivers the same parcel twice. The shopkeeper checks the parcel's seal number against her logbook: already received → she sets it aside instead of paying for it again.
Exactly-once impossibility (Two Generals)	Two captains across a foggy valley must attack together, sending runners who may be caught. Each runner needs a runner back confirming receipt, then a runner confirming *that*... forever. No number of runners makes both certain — so they plan for "the order might arrive twice or not at all."
Partition-by-key / ordering	The depot sorts all of one household's parcels onto a single shelf so they're delivered in the order sent; different households' shelves are worked in parallel. You never need *the whole town* in one order — just each household's parcels in order.
Global ordering kills parallelism	Insisting *every* parcel in town be delivered in strict order means one courier doing everything single-file — the other five stand idle. Per-household order lets all six work at once.
Dead Letter Queue	The depot's "undeliverable" bin: a parcel with a smudged address that's failed delivery five times is set aside there so the rest of the round flows, and a supervisor investigates it later.
Poison message	A parcel rigged to jam the sorting machine. Without an "undeliverable" bin, it jams the line forever and nothing behind it moves. The bin gets it out of the way.
Dual-write / Outbox	You must both record a sale in your shop's ledger *and* notify the depot. Instead of two risky separate acts, you write the sale *and* an outgoing-notice in the same ledger stroke; a clerk later reads the outgoing-notices and walks them to the depot — so a power cut can never leave one done without the other.
Saga	A multi-shop layaway: the tailor cuts cloth, the dyer colours it, the seamstress sews. If the dyer's vat spills, each shop has a written *undo* note — the tailor un-cuts (returns the bolt), and the layaway unwinds cleanly instead of leaving a half-made coat nobody paid for.
Schema registry	The town's standard parcel-label format kept at the post office. Before printing new labels, you check them against the standard; old depots can still read new labels (they ignore the extra fields), so nobody's deliveries break when the format evolves.
Consumer lag	The height of the depot's unworked-parcel pile. A pile that's steady during a rush is fine; a pile that keeps *growing* means you need more couriers before customers start complaining.

17 Memory Tricks / Mnemonics

17.1 Three shapes: "QPL --- Queue, Publish, Log"

```
Q ueue  → to-do list, ONE worker, delete-on-ack       (RabbitMQ, SQS)
P ub/sub→ loudspeaker, EVERY subscriber gets a copy    (SNS, fanout)
L og    → ledger, retained + replayable, own offset    (Kafka, Kinesis)
```

17.2 Delivery semantics: "0-1-∞"

```
At-MOST-once  (0 or 1) → may LOSE,     never dup → commit BEFORE process, acks=0
At-LEAST-once (1+)     → never lose,    may DUP   → commit AFTER process, acks=all  ☑default
Exactly-once  (1)      → DELIVERY impossible (Two Generals) → at-least-once + idempotent
```

17.3 Ack timing: "Commit After = At-least-once"

```
commit BEFORE process → crash = LOSS   (at-most-once)
commit AFTER  process → crash = DUP    (at-least-once)  → idempotency saves you
```

17.4 Kafka durability triad: "3, 2, all"

```
Replication.factor = 3
Min.insync.replicas = 2
Acks = all
→ survives 1 broker down, zero loss
```

17.5 Kafka invariant: "One partition, one consumer (per group)"

```
partition = unit of PARALLELISM + ORDERING
#consumers > #partitions → idle consumers
ordering only WITHIN a partition → partition by KEY
```

17.6 RabbitMQ exchanges: "D-T-F-H"

Python

```python
Direct  = exact key match
Topic   = wildcard (* one word, # many)
Fanout  = broadcast (ignore key)
Headers = match on attributes
```

17.7 Reliability checklist: "I-D-O-S" (say all four, every design)

```
I dempotent consumers (dedup table)
D LQ + retry with backoff + jitter (poison)
O utbox + CDC (dual-write)
S aga (distributed transaction)
```

17.8 Outbox vs Inbox

Python

```python
OUTBOX = produce-once-effectively  (write event in same tx as business data)
INBOX  = consume-once-effectively  (record messageId in same tx as side effect)
```

17.9 Stream-processing hard part: "Event time + Watermark"

```
window by EVENT time, not arrival time
WATERMARK = "I've seen everything up to T" → close window; later = LATE
```

18 Common Interview Questions

18.1 Q1: When would you use a message queue instead of a synchronous API call?

Model answer: "Use async messaging when the work can happen *after* you respond to the user, and you want to decouple availability and latency. Three triggers: (1) **load leveling** — the producer is bursty and a queue smooths spikes so the consumer (and DB) aren't overwhelmed; (2) **decoupling** — you want to add consumers without touching the producer, or one event has many independent consumers; (3) **resilience** — if a downstream is down, the queue buffers instead of failing the user's request. Keep it synchronous when the caller genuinely needs the answer to proceed — a price quote, an auth check, an inventory-availability read. The tell is: 'does the user need this result before we can respond?' If no → async."

Follow-ups:

- *What's the cost of going async?* → Eventual consistency, duplicate handling, ordering concerns, operational complexity (DLQs, lag monitoring), and latency under load.
- *How does availability change?* → Sync chains multiply availability ($0.999^4 \approx 99.6\%$); async makes the producer's availability independent of consumers.

18.2 Q2: Explain the difference between a queue, pub/sub, and a log.

Model answer: "A **queue** is point-to-point: each message goes to exactly one of the competing consumers and is deleted on ack — a to-do list (RabbitMQ, SQS). **Pub/sub** is fan-out: each subscriber gets its own copy of every message — a broadcast (SNS, fanout exchange). A **log** is append-only and retained: consumers track their own offset, reading is non-destructive, so you get replay, multiple independent consumer groups, and event sourcing (Kafka, Kinesis). The defining axis is *delete-on-read* (broker-centric) vs *retained-and-replayable* (log-centric). The moment a requirement mentions replay or multiple independent downstreams, choose a log."

Follow-ups:

- *Kafka does both queue and pub/sub — how?* → Within one consumer group, partitions split across consumers (queue behavior); across groups, every group sees everything (pub/sub) — plus retention.
- *When RabbitMQ over Kafka?* → Complex routing, per-message TTL/priority/delay, RPC patterns, lower-throughput task queues.

18.3 Q3: Why is exactly-once delivery impossible, and what do you do instead?

Model answer: "It's the Two Generals Problem. After a consumer processes a message it must ack the broker, but the ack can be lost — by a network drop or a crash right after processing. The broker then can't tell 'processed, ack lost' from 'crashed before processing.' Its only options are redeliver (risk duplicate) or not (risk loss); there's no third choice, so exactly-once *delivery* is impossible. The practical answer is **at-least-once delivery + idempotent consumers**: commit the offset *after* processing (never lose), and make processing dedup duplicates — typically a dedup/inbox table keyed by messageId, written in the same transaction as the side effect. That gives exactly-once *processing*, which is what people actually want. Kafka transactions give true exactly-once, but only Kafka-to-Kafka — the moment you write to an external DB or call an API, you're back to idempotency."

Follow-ups:

- *Show the dedup code.* → Insert messageId with a unique constraint in the same tx as the business write; UniqueViolation → no-op.
- *What about double-charging a card?* → Two layers: your dedup table *and* the gateway's idempotency key.

18.4 Q4: How does Kafka guarantee ordering, and what are the limits?

Model answer: "Ordering is guaranteed **only within a partition** — each partition is an append-only log. There's no global order across a topic, because that would require a single serialization point and kill parallelism. To get the ordering you need, you partition by **key**: all events for one entity (orderId, accountId) hash to the same partition and are strictly ordered relative to each other, while different entities parallelize across partitions. You almost never need global order — you need per-entity order, and partition-by-key gives exactly that. The catch: ordering breaks if you increase partition count (keys rehash) or run more than one in-flight request without idempotence (retries can reorder)."

Follow-ups:

- *If you truly need global order?* → One partition, one consumer — no horizontal scaling. Question whether the requirement is real.
- *How do you handle out-of-order events?* → Version numbers / last-write-wins, or event-time buffering with watermarks.

18.5 Q5: What is the dual-write problem and how do you solve it?

Model answer: "A service often needs to update its DB *and* publish an event atomically, but the DB and broker are separate systems — you can't put both in one transaction. So a crash between them leaves either a committed order with no event (lost event) or a published event for an order that rolled back (phantom event). Wrapping them in either order can't fix the crash window. The solution is the **transactional outbox**: in the same DB transaction as the business write, insert a row into an `outbox` table. A separate relay — a poller or, better, CDC (Debezium tailing the WAL) — reads the outbox and publishes to the broker at-least-once. Because the order and outbox row commit together, you can't have one without the other; consumers dedup on messageId to absorb the at-least-once relay."

Follow-ups:

- *Poll vs CDC?* → CDC has no polling lag, no extra DB query load, and reads straight from the WAL — preferred at scale.
- *What's the consumer-side mirror?* → The inbox pattern: record messageId in the same tx as the side effect.

18.6 Q6: Walk me through SQS visibility timeout and DLQs.

Model answer: "When a consumer receives an SQS message, SQS hides it from other consumers for the **visibility timeout** (default 30s). The consumer processes and calls `DeleteMessage` (the ack). If it deletes in time, done. If it crashes or runs long, the timeout expires, the message reappears, and another consumer gets it — that's how SQS achieves at-least-once. You set the timeout above your p99 processing time, or extend it dynamically with `ChangeMessageVisibility` for long jobs. For poison messages, you configure a **redrive policy** with `maxReceiveCount`: after, say, 5 receives, SQS automatically moves the message to a **DLQ** for inspection and replay, so it stops cycling through your workers. Always use **long polling** (`WaitTimeSeconds=20`) to avoid wasteful empty receives."

Follow-ups:

- *Standard vs FIFO?* → Standard: best-effort order, at-least-once, huge throughput. FIFO: strict order per MessageGroupId, exactly-once processing (5-min dedup), 300/s per group.
- *Why SNS→SQS rather than SNS→Lambda?* → The SQS buffer gives each consumer durability and back-pressure; one slow consumer doesn't affect others.

18.7 Q7: Design a notification system that sends email, SMS, and push for an event.

Model answer: "One event, three independent channels with different rate limits and failure profiles → **SNS→SQS fan-out**. The producer publishes `NotifyUser` once to an SNS topic; SNS delivers a copy to three SQS queues (email, sms, push), each consumed by its own worker pool. Each queue is an independent buffer with its own retry and DLQ, so if the SMS provider is down, its queue backs up without touching email. Workers are **idempotent** — dedup on `(userId, eventId, channel)` so a redelivery doesn't send a second email. Retries use **exponential backoff + jitter**; after max attempts → DLQ + alert. For provider rate limits, the pull-based SQS consumers throttle naturally (back-pressure). If I needed replay or analytics on notifications, I'd put Kafka in front instead of/alongside SNS."

Follow-ups:

- *Ordering?* → Usually irrelevant across channels; if a user mustn't get 'shipped' before 'confirmed,' partition by userId in Kafka.
- *Dedup window?* → Keep processed eventIds as long as the message could be redelivered (retention + visibility timeout).

18.8 Q8: What is a poison message and how do you handle it?

Model answer: "A poison message is one that can never be processed successfully — corrupt payload, schema mismatch, or a referenced entity that no longer exists. The danger is head-of-line blocking: if you retry it forever (or `requeue=true` in RabbitMQ), it loops, pins a worker, and in an ordered partition blocks everything behind it. The fix is a **retry cap + DLQ**: retry transient failures a bounded number of times with exponential backoff and jitter, and after the cap, move the message to a dead-letter queue. That unblocks the stream, preserves the bad message for debugging, and lets ops fix and replay it. Distinguishing transient (retry) from permanent (DLQ immediately) failures in code avoids wasting retries on hopeless messages."

Follow-ups:

- *RabbitMQ specifics?* → `basic.nack(requeue=false)` → DLX; never `requeue=true` on poison (hot loop).
- *Kafka specifics?* → Retry-topic chain + a dead-letter topic; skip the offset past it if needed.

18.9 Q9: Compare Kafka and RabbitMQ. When would you pick each?

Model answer: "RabbitMQ is a **smart broker, dumb consumer** — routing logic (direct/topic/fanout/headers exchanges) lives in the broker, messages are deleted on ack, and you get per-message control: TTL, priority, delayed delivery, requeue. Kafka is a **dumb broker, smart consumer** — the broker just appends to a partitioned log; consumers own their offsets, data is retained and replayable, and ordering is per-partition. Pick **RabbitMQ** for complex routing, task queues, RPC patterns, per-message priorities/TTL, and moderate throughput. Pick **Kafka** for event streaming, replay, event sourcing, CDC, many independent consumer groups, stream processing, and extreme throughput (millions/s via sequential disk, zero-copy, batching). If a requirement says 'replay history' or 'three teams independently consume the same stream,' that's Kafka."

Follow-ups:

- *Why is Kafka so fast?* → Sequential disk writes + page cache, zero-copy sendfile, batching, compression.
- *Does RabbitMQ scale?* → Yes (clustering, quorum queues), but its model favors routing flexibility over the raw throughput and retention Kafka gives.

18.10 Q10: What's the difference between at-least-once and exactly-once, and how do you make a consumer idempotent?

Model answer: "At-least-once never loses but may duplicate (commit offset after processing, producer retries on ack failure). Exactly-once delivery is impossible, so we approximate it with at-least-once + idempotency = exactly-once *processing*. To make a consumer idempotent: (1) a **dedup/inbox table** — insert the messageId with a unique constraint in the same transaction as the side effect; a duplicate hits the constraint and no-ops; (2) **natural idempotency** — use set-operations (`status='PAID'`) or upserts instead of increments/inserts; (3) **conditional writes** — `UPDATE ... WHERE version = N` so a stale duplicate no-ops. The dedup row and the business effect must commit atomically, or a crash between them reintroduces loss or duplicates."

Follow-ups:

- *How long do you keep dedup keys?* → As long as a redelivery is possible (retention + visibility/redelivery window), then TTL them.
- *Kafka exactly-once?* → Idempotent producer (PID+sequence) + transactions (atomic consume-process-produce with `read_committed`), Kafka-to-Kafka only.

18.11 Q11: How do consumer groups and rebalancing work in Kafka, and what's the operational risk?

Model answer: "A consumer group shares a topic's partitions — each partition is owned by exactly one consumer in the group, so the group's max parallelism is the partition count. When a consumer joins or leaves (deploy, crash, scale), Kafka **rebalances**: reassigns partitions. Old eager rebalancing was stop-the-world — every consumer revoked everything and processing halted; **cooperative/incremental** rebalancing (2.4+) only moves the partitions that must move. The operational risk is **rebalance storms**: a slow consumer that exceeds `max.poll.interval.ms` gets evicted, triggering a rebalance, redelivery (duplicates), and a latency spike — repeatedly. Mitigate with cooperative rebalancing, static membership (`group.instance.id`) to survive rolling restarts, tuning `max.poll.records`, and moving heavy work off the poll thread. Monitor **consumer lag** to know if you need more consumers (up to #partitions)."

Follow-ups:

- *More consumers than partitions?* → Extras sit idle. Add partitions to scale further (accepting key remap).
- *Auto vs manual offset commit?* → Manual commit after processing for at-least-once; auto-commit can advance past unprocessed messages (silent loss).

18.12 Q12: Explain event time vs processing time and watermarks in stream processing.

Model answer: "Event time is when the event actually happened (a timestamp in the event); processing time is when your system handles it. They diverge due to network delays, retries, offline buffering, and out-of-order delivery — a mobile click at 12:00:03 might arrive at 12:00:09. Windowing by processing time puts late events in the wrong bucket, corrupting analytics, so correct systems window by **event time**. But then you need to decide when an event-time window is 'complete,' since late events can still trickle in. A **watermark** is a heuristic — 'I believe I've seen all events up to event-time T.' When the watermark passes a window's end, the window closes and emits; events arriving after are 'late' and handled by policy (drop, side-output, or update with allowed lateness). Fault tolerance for the windowed state comes from changelog topics (Kafka Streams) or checkpoints (Flink)."

Follow-ups:

- *Window types?* → Tumbling (fixed, non-overlapping), sliding/hopping (overlapping), session (activity-gap).

- *Kafka Streams vs Flink?* → Streams is an embedded library (Kafka-to-Kafka, RocksDB state); Flink is a cluster with richer event-time/state and checkpoint-based exactly-once.

19 Senior-Level Discussion Points

19.1 The Exactly-Once Myth (system level)

Kafka markets "exactly-once semantics," but it's exactly-once *within the Kafka boundary*: idempotent producer dedups retries, transactions make consume-process-produce atomic, `read_committed` hides aborted records. The instant a consumer writes to an external system (SQL, Redis, an HTTP API, an email), Kafka transactions no longer cover it — you're at-least-once across that boundary and *must* use idempotent consumers. The only way to extend exactly-once outward is to enroll the external write and the offset commit in *one* transactional store (e.g., write output and committed offset to the same Postgres in one tx). Always clarify "exactly-once *processing*, end-to-end via idempotency" vs "Kafka's internal exactly-once."

19.2 Throughput vs Latency Tuning

`linger.ms` and `batch.size` are the explicit dial: `linger.ms=0` minimizes latency but sends tiny batches (more requests, less compression, lower throughput); `linger.ms=20+batch.size=64KB` maximizes throughput at the cost of up to 20ms added latency. Compression (`zstd`/`lz4`) trades CPU for network/storage and *improves* with bigger batches. There's no universal setting — a clickstream pipeline wants throughput; a fraud-blocking path wants latency.

19.3 Choosing Partition Count Is a One-Way-ish Door

Too few partitions caps consumer parallelism; too many costs file handles, memory, longer rebalances, and worse latency. Increasing partitions later **rescrambles key→partition mapping**, breaking per-key ordering for in-flight data and any compaction assumptions. So you over-provision modestly up front (e.g., 2–3× current need) rather than constantly resizing. This is a classic "explain the trade-off" senior probe.

19.4 Quorum Queues and Broker Durability

RabbitMQ classic mirrored queues had data-loss edge cases under partitions; **quorum queues** (Raft-based) are the modern durable choice — analogous to Kafka's ISR. Knowing that messaging brokers themselves are distributed systems (replication, leader election, split-brain) — and that `acks`/`min.insync.replicas`/`unclean.leader.election` are exactly the CAP/quorum dials from distributed systems — ties the whole topic together.

19.5 Back-Pressure vs Buffering Forever

Load leveling only works for *transient* spikes. A *sustained* overload turns the queue into an unbounded buffer that eventually exhausts disk and takes the broker (and everyone) down. The senior insight: a queue is not a substitute for capacity — it's a shock absorber. You need back-pressure (lag alarms, pull-based consumers, publisher rejection propagating 503s to clients) and autoscaling of consumers tied to lag.

19.6 Ordering vs Idempotency vs Delivery Semantics Are Orthogonal

Candidates conflate them. Ordering is "in what sequence" (per-partition). Idempotency is "safe to reprocess" (dedup). Delivery semantics is "how many times" (at-least-once). You can have ordered-but-duplicated, unordered-but-deduped, etc. A complete design addresses all three explicitly.

19.7 Schema Evolution Is a Production Outage Waiting to Happen

Producer and consumer deploy on different schedules. A breaking schema change (renaming/removing a field, changing a type) deserializes into garbage or exceptions on the lagging side. A schema registry with enforced **backward** compatibility (so new consumers read old data) and **forward** compatibility (so old consumers tolerate new data) is what lets teams deploy independently — the entire point of decoupling.

20 Typical Mistakes Candidates Make

Mistake	Correction
”I'll use a queue” without specifying queue vs log	They have different semantics (delete-on-ack vs retained/replayable). Name which and why.
”Kafka guarantees exactly-once” (full stop)	Only within Kafka. External writes need idempotent consumers. Exactly-once *delivery* is impossible (Two Generals).
Treating exactly-once delivery as achievable	It's provably impossible over a lossy channel. Do at-least-once + idempotency = exactly-once *processing*.
Committing the offset / acking *before* processing	That's at-most-once — a crash loses the message. Commit *after* processing for at-least-once.
Forgetting the **dual-write problem**	DB + broker can't share a transaction → lost/phantom events. Use the Outbox + CDC.
No idempotency on consumers	At-least-once *will* deliver duplicates. Dedup table / natural idempotency / conditional writes are mandatory.
Expecting global ordering from Kafka	Ordering is per-partition only. Partition by key for per-entity order; global order kills parallelism.
`requeue=true` on every failure (RabbitMQ)	Poison messages hot-loop forever. Use bounded retries with backoff → DLX.
No DLQ / no retry strategy	Poison messages block the stream (head-of-line). Always: bounded retries + backoff + jitter → DLQ.
Retrying with no backoff/jitter	Hammers a struggling downstream and synchronizes retries into a new spike. Exponential backoff + jitter.
Thinking a queue gives infinite capacity	It absorbs *spikes*, not sustained overload. Need back-pressure + consumer autoscaling on lag.
Ignoring schema evolution	Producer/consumer drift breaks deserialization. Schema registry + compatibility checks.
More consumers than partitions for ”more speed”	Extra consumers sit idle. Parallelism is capped by partition count.
Windowing by processing time	Late events land in wrong buckets. Window by event time; use watermarks for completeness.

21 How This Connects to Other Topics

21.1 Distributed Systems

- Brokers *are* distributed systems: Kafka ISR/leader-election = quorum + consensus; `acks`/`min.insync.replicas` = the CAP/durability dial; `unclean.leader.election` = the AP-vs-CP choice (availability vs zero loss).
- Saga = the distributed-transaction alternative to 2PC (eventual consistency, compensations,

non-blocking).
- At-least-once + idempotency is the same "effectively-once" idea from distributed systems.

21.2 System Design

- The message queue is a core box in nearly every design diagram (between app servers and workers). Async fan-out enables the **fan-out-on-write** feed model, decoupled microservices, and CQRS/event-sourcing read models.
- Load leveling protects the DB tier; back-pressure connects to rate limiting and circuit breakers.

21.3 Databases

- The Outbox pattern lives *in* the database; CDC streams the WAL/binlog into Kafka. Log compaction turns a topic into a materialized changelog (a DB-like state store).
- Idempotency relies on DB constraints (unique keys) and transactions; exactly-once processing requires the offset and the side effect in one transactional store.

21.4 Concurrency

- Consumer groups = work distribution across threads/processes; prefetch/QoS = bounded in-flight work = the producer-consumer concurrency pattern at distributed scale.
- Back-pressure mirrors bounded buffers and Reactive Streams credit-based flow control.

21.5 Performance Engineering

- Kafka's speed = sequential I/O + page cache + zero-copy + batching — the same mechanical-sympathy principles as systems performance.
- `linger.ms`/`batch.size`/compression are throughput-vs-latency tuning knobs.

21.6 APIs / Networking

- Idempotency keys (Stripe-style `Idempotency-Key` header) make retried HTTP POSTs safe — the same dedup idea at the API edge.
- Webhooks are async fan-out over HTTP; at-least-once delivery means webhook receivers must be idempotent too.

22 FAANG Interview Tips

In system-design rounds:

1. **Decide sync vs async explicitly.** "Does the user need this result to respond? If not, push it async." Justify each async hop.
2. **Pick the shape out loud:** "I need replay and multiple independent consumers, so this is a log — Kafka — not an SQS queue."
3. **Volunteer the failure modes** before being asked: duplicates → idempotent consumers; dual-write → outbox; poison → DLQ+backoff; ordering → partition by key; back-pressure → lag alarms. This is the senior differentiator.
4. **Quantify:** partition count from peak QPS / per-consumer rate; spike-absorption backlog and drain time; `acks=all` + `min.insync.replicas=2` + RF=3 for durability.
5. **Name the trade-off** every time: throughput vs latency (`linger.ms`), ordering vs parallelism (global vs per-key), consistency vs availability (`unclean.leader.election`).

In depth/knowledge rounds:

1. Know Kafka cold: partitions (parallelism+ordering), consumer groups + rebalancing, offsets (committed vs current, manual commit), ISR + leader election + `acks`/`min.insync.replicas`, retention vs compaction, idempotent producer + transactions, consumer lag.
2. Be able to *derive* the Two Generals impossibility, then pivot to at-least-once + idempotent dedup-table code.
3. Be able to draw the Outbox + CDC flow and explain why naive publish-then-commit fails.

Communication patterns:

- "The classic risk here is the dual-write problem; the standard fix is a transactional outbox with CDC."
- "I'd partition by orderId so each order's events are ordered, while different orders parallelize — we don't need global order."
- "This is at-least-once, so the consumer must be idempotent — I'll dedup on messageId in the same transaction as the write."

Red flags to avoid: claiming exactly-once delivery; "just put it on a queue" with no duplicate/ordering/DLQ story; confusing queue vs log; ignoring the dual-write problem; expecting global ordering from Kafka.

23 Revision Cheat Sheet

23.1 10-Minute Summary

1. **Why async:** decoupling (producer doesn't know consumers), load leveling (absorb spikes), resilience (buffer downstream failures). Cost: eventual consistency, duplicates, ordering care, ops.
2. **Three shapes:** Queue (one consumer, delete-on-ack — RabbitMQ/SQS), Pub/Sub (every subscriber a copy — SNS), Log (retained, replayable, offsets — Kafka/Kinesis).
3. **Broker vs log:** delete-on-read vs retained-and-replayable. Replay/multiple-consumers/event-sourcing → log.
4. **Delivery:** at-most-once (commit before process, may lose), **at-least-once** (commit after process, may dup — the default), exactly-once delivery = impossible (Two Generals).
5. **Effectively-once:** at-least-once + **idempotent consumer** (dedup table keyed by messageId, in same tx as side effect).
6. **Kafka exactly-once:** idempotent producer (PID+seq) + transactions (atomic consume-process-produce, `read_committed`). Kafka-to-Kafka only.
7. **Kafka core:** partition = parallelism + ordering unit; partition by **key** for per-entity order; consumer group (one partition → one consumer); offset (manual commit after processing); ISR + leader election; durability triad RF=3 / `min.insync.replicas=2` / `acks=all`; retention vs **compaction**; **lag** = key health metric.
8. **RabbitMQ:** publish to **exchange** (direct/topic/fanout/headers) → bindings → queues; ack/-nack/reject; **prefetch (QoS)** = back-pressure; **DLX** + TTL + priority; publisher confirms.
9. **SQS/SNS/Kinesis:** SQS = queue + visibility timeout + long polling + DLQ (standard vs FIFO); SNS = fan-out (SNS→SQS pattern); Kinesis = managed log (shards, replay).
10. **Ordering:** per-partition only; partition by key; global order kills parallelism; handle out-of-order with versions/watermarks.
11. **Reliability (I-D-O-S):** Idempotent consumers; DLQ + retry w/ backoff + jitter (poison); Outbox + CDC (dual-write); Saga (distributed tx, choreography vs orchestration). Plus schema registry for evolution.
12. **Stream processing:** Kafka Streams (library) vs Flink (cluster); tumbling/sliding/session windows; **event time** not processing time; **watermarks** for window completeness.

23.2 Key Comparison Tables

Delivery semantics:

```
at-most-once : commit BEFORE process, acks=0   → may LOSE, no dup
at-least-once: commit AFTER  process, acks=all → no loss, may DUP  ⎚ + idempotency
exactly-once : DELIVERY impossible → at-least-once + idempotent = exactly-once PROCESSING
```

Queue vs Log:

```
Queue (RabbitMQ/SQS): delete-on-ack, no replay, routing-rich, one consumer set
Log   (Kafka/Kinesis): retained, replayable, offsets, many groups, per-partition order
```

Kafka durability:

```
RF=3 + min.insync.replicas=2 + acks=all → survive 1 broker down, zero loss
unclean.leader.election=false → reject writes rather than lose data (CP)
```

RabbitMQ exchanges:

```python
direct=exact key | topic=wildcard(*,#) | fanout=broadcast | headers=attributes
```

Reliability patterns:

```
Idempotent consumer (dedup table)  | DLQ + backoff + jitter (poison)
Outbox + CDC (dual-write)          | Saga (distributed tx: choreography vs orchestration)
```

23.3 Most Important Concepts (Priority Order)

1. Sync-vs-async decision + decoupling/load-leveling/back-pressure
2. Queue vs pub/sub vs log semantics (and broker vs log)
3. Delivery semantics + Two Generals + at-least-once + idempotent consumer (with dedup code)
4. Kafka deep dive (partitions, consumer groups, offsets, ISR/acks, retention/compaction, lag)
5. Ordering: per-partition only + partition-by-key
6. Dual-write problem + Outbox + CDC
7. DLQ + retry/backoff/jitter + poison messages
8. RabbitMQ exchanges + prefetch + DLX
9. SQS/SNS/Kinesis (visibility timeout, fan-out, shards)
10. Saga (choreography vs orchestration)
11. Schema registry + compatibility
12. Stream processing (windowing, event time, watermarks)

Study tip: for every concept, be able to (1) define it in one sentence, (2) name a real system that uses it, (3) state the trade-off, and (4) describe a failure scenario and its mitigation. If you can do that for "exactly-once," "ordering," "dual-write," and "back-pressure," you can carry any message-queue interview.